

ACTIVITY GUIDE ON CONTAINER WITH KUBERNETES

TABLE OF CONTENTS

PRACTICAL LESSON 24: INGRESS ROUTING AND CONTROLLER

Practical Lesson 24: Overview	4
Practical Lesson 24: Configure Horizontal Pod Autoscaler.....	5

PRACTICAL LESSON 25: VOLUME STORAGE AND CLASSES

Practical Lesson 25: Overview	16
Practical Lesson 25.1: Configure a Pod to use Volume for storage	17
Practical Lesson 25.2: Create a Kubernetes Job	21

PRACTICAL LESSON 26: NAMESPACES AND RESOURCE QUOTAS

Practical Lesson 26: Overview	25
Practical Lesson 26.1: Assign memory Resources to Containers and Pods.....	26
Practical Lesson 26.2: Assign CPU Resources to Containers and Pods.....	32
Practical Lesson 26.3: Assign Pod to a Node	37

PRACTICAL LESSON 27: OKE ARCHITECTURE

Practical Lesson 22: Overview	41
-------------------------------------	----

PRACTICAL LESSON 24

INGRESS ROUTING AND CONTROLLER

Practical Lesson 24: Overview

Practical Overview

Dynamic pod scaling is a Kubernetes feature that makes it easy to control resource utilization and ensure that an adequate number of pods are running

Estimated Completion Time: 50mins

Practical Lesson 24.1: Configure Horizontal Pod Autoscaler

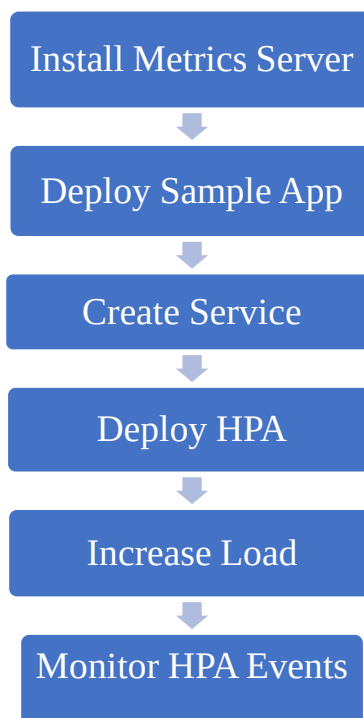
Overview

Horizontal pod autoscaling allows Kubernetes to deal with increased or decreased loads by scaling the number of pod replicas related to a specific deployment or stateful set. While vertical scaling means assigning more resources to the running pods, horizontal scaling creates or removes pods within the same resource allocation

Prerequisites

- Should be connected to the internet
- OKE cluster
- Docker hub account

Tasks



1. INSTALL METRICS SERVER

Follow the steps for installing the metrics server

Metrics Server to monitor CPU and Memory resources in Kubernetes Cluster.

- Login to OCI
- Open the code shell
- To Deploy metrics server:
`git clone https://github.com/kubernetes-sigs/metrics-server`
- For creating the metrics server: `kubectl apply -k ./metrics-server/manifests/base/`

```
[root@dlp ~]# kubectl apply -k ./metrics-server/manifests/base/
serviceaccount/metrics-server created
clusterrole.rbac.authorization.k8s.io/system:aggregated-metrics-read created
clusterrole.rbac.authorization.k8s.io/system:metrics-server created
rolebinding.rbac.authorization.k8s.io/metrics-server-auth-reader created
clusterrolebinding.rbac.authorization.k8s.io/metrics-server:system:aggregated-metrics-read created
clusterrolebinding.rbac.authorization.k8s.io/system:metrics-server created
service/metrics-server created
deployment.apps/metrics-server created
apiservice.apiregistration.k8s.io/v1beta1.metrics.k8s.io created
```

Execute `kubectl get pods -n kube-system`

NAME	READY	STATUS	RESTARTS	AGE
coredns-558bd4d5db-f8cvh	1/1	Running	4	16h
coredns-558bd4d5db-tg8v6	1/1	Running	3	16h
etcd-dlp.srv.world	1/1	Running	3	16h
kube-apiserver-dlp.srv.world	1/1	Running	3	16h
kube-controller-manager-dlp.srv.world	1/1	Running	3	16h
kube-flannel-ds-5vvnv2	1/1	Running	3	16h
kube-flannel-ds-lf65f	1/1	Running	3	16h
kube-flannel-ds-wdvpn	1/1	Running	4	16h
kube-proxy-mqs8c	1/1	Running	3	16h
kube-proxy-wzh69	1/1	Running	3	16h
kube-proxy-zjzzf	1/1	Running	3	16h
kube-scheduler-dlp.srv.world	1/1	Running	3	16h
metrics-server-6f87b5f98d-dr6g7	0/1	Running	0	14s

[metrics-server] pod has been deployed

To display CPU and Memory resources, run as follows.

```
[root@dlp ~]# kubectl top node
```

NAME	CPU(cores)	CPU%	MEMORY(bytes)	MEMORY%
dlp.srv.world	187m	2%	1292Mi	8%
node01.srv.world	181m	9%	412Mi	11%
node02.srv.world	38m	1%	371Mi	10%

You have successfully installed the Metrics Server

2. DEPLOY A SAMPLE APP

The following section shows how to create a Docker image for a small PHP app that performs a resource-intensive calculation.

1. Create the app directory:

```
mkdir test-app
```

2. Create a Dockerfile in a text editor:

nano Dockerfile

You can work in the code shell or with the code editor

3. Open the code editor, navigate to the test-app folder and create a Dockerfile.

```
FROM php:5-apache

COPY index.php
/var/www/html/index.php

RUN chmod a+rx index.php
```

4. Create an index.php file:

Add the mathematical operation to the file to create a CPU load:

```
<?php

$x = 0.0001;

for ($i = 0; $i <= 1000000; $i++) {

    $x += sqrt($x);

}

echo "OK!";

?>
```

5. Build the Docker image:

```
docker build -t test-app:v1 .
```

6. Tag the image. Use your Docker Hub account username:

```
docker image tag test-app:v1 [your-docker-hub-username]/test-app:v1
```



```
docker image tag test-app:v1 paulmausomi/test-app:v1
```

where, paulmausomi is the docker hub username

7. Before pushing into the docker hub, execute the above command

```
docker login
```

After executing, it will ask for your username and password

8. Push the image to Docker Hub by

```
docker image push [your-docker-hub-username]/test-app:v1
```

9. Create a deployment YAML in the code editor and let the file name be hpa-test.yaml

10. The YAML defines the deployment and the service that exposes it. The **spec.template.spec.containers** section specifies that the deployment uses the Docker image created in the previous steps. Furthermore, the **resources** sub-section contains resource limits and requests.

3. CREATE A SERVICE

Here the snippet shows a Deployment and a Service object.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hpa-test
spec:
  selector:
    matchLabels:
      run: hpa-test
  replicas: 1
  template:
    metadata:
      labels:
        run: hpa-test
    spec:
      containers:
        - name: hpa-test
          image: paulmausomi/test-app:v1
          ports:
            - containerPort: 80
          resources:
            limits:
              cpu: 500m
            requests:
              cpu: 200m
---
apiVersion: v1
kind: Service
metadata:
  name: hpa-test
  labels:
    run: hpa-test
spec:
  ports:
    - port: 80
  selector:
    run: hpa-test
```

11. Create the deployment by using the **kubectl apply** command:

```
kubectl apply -f hpa-test.yaml
```

The output will create a deployment and service object.

12. Confirm that the objects are ready and running.

```
kubectl get all
```

4. Deploy HPA

With the deployment up and running, proceed to create a HorizontalPodAutoscaler object. The sections below illustrate the two methods for creating HPAs.

CREATE HPA VIA KUBECTL COMMAND

Create an HPA using the `kubectl autoscale` command.

```
kubectl autoscale deployment hpa-test --cpu-percent=50 --min=1 --max=10
```

The `--cpu-percent` flag sets the CPU utilization percentage limit that triggers the autoscaler. The `--min` and `--max` flags specify the minimal and maximal number of pod replicas, i.e., the range within which the HPA is allowed to scale.

CREATE HPA VIA YAML MANIFEST

Alternatively, create the HPA declaratively by defining it in a YAML file.

1. Create the *hpa.yaml* file:
2. The YAML contains `spec.minReplicas`, `spec.maxReplicas`, and `targetCPUUtilizationPercentage` fields that correspond to the `--min`, `--max`, and `--cpu-percent` flags in the command-line method for creating HPAs.

```
apiVersion: autoscaling/v1
kind: HorizontalPodAutoscaler
metadata:
  name: hpa-test
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: hpa-test
  minReplicas: 1
  maxReplicas: 10
  targetCPUUtilizationPercentage: 50
```

Create the HPA with **kubectl apply**:

```
kubectl apply -f hpa.yaml
```

You can either Deploy HPA via the `kubectl` command or you can create a YAML file.

5. INCREASE THE LOAD

1. Before increasing the load to see the HPA in action, use **kubectl get** to check the status of the controller.

```
kubectl get hpa
```

The **TARGETS** column in the output shows that no load is generated. Consequently, the number of created pod replicas is still one.

```
ritesh@codeeditor:test-app (us-ashburn-1)$ kubectl get hpa
NAME          REFERENCE          TARGETS  MINPODS  MAXPODS  REPLICAS  AGE
hpa-test      Deployment/hpa-test  0%/50%   1         10        1          37s
ritesh@codeeditor:test-app (us-ashburn-1)$ kubectl get hpa
NAME          REFERENCE          TARGETS  MINPODS  MAXPODS  REPLICAS  AGE
hpa-test      Deployment/hpa-test  0%/50%   1         10        1          63s
```

2. Now increase the CPU load using the load generator. Execute the following command in a new terminal tab/window:

```
kubectl -n default run -i --tty load-generator -
-rm --image=busybox --restart=Never -- /bin/sh -
c "while sleep 0.01; do wget -q -O- http://hpa-
test; done"
```

The command uses the *busybox* image to generate load and repeatedly query the *hpa-test* deployment. The deployment performs mathematical operations and engages the CPU.

3. In the first terminal tab/window, type the following command to watch the status of the HPA:

```
kubectl get hpa -w
```


LESSON 25

VOLUME STORAGE AND CLASSES

Practical Lesson 25: Overview

Practical Overview

In this practical lesson, you will learn about how to use volumes in Kubernetes, which will help you to have persistent data if your pod is terminated and you will also learn how to work with jobs for repetitive works.

Estimated Completion Time: 50mins

Practical Lesson 25.1: Configure a Pod to use Volume for storage

Overview

A Container's file system lives only as long as the Container does. So when a Container terminates and restarts, filesystem changes are lost. For more consistent storage that is independent of the Container, you can use a Volume. This is especially important for stateful applications, such as key-value stores (such as Redis) and databases

Prerequisites

- Should be connected to the internet
- OKE cluster
- Docker hub account

Tasks

1. In this exercise, you create a Pod that runs one Container. This Pod has a Volume of type emptyDir that lasts for the life of the Pod, even if the Container terminates and restarts.

Navigate to the folder and create a file

Here is the configuration file for the Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: redis
spec:
  containers:
  - name: redis
    image: redis
    volumeMounts:
    - name: redis-storage
      mountPath: /data/redis
  volumes:
  - name: redis-storage
    emptyDir: {}
```

1. Create the Pod:

```
kubectl apply -f redis.yaml
```

2. Verify that the Pod's Container is running, and then watch for changes to the Pod:

```
kubectl get pod redis --watch
```

The output looks like this:

NAME	READY	STATUS	RESTARTS	AGE
redis	1/1	Running	0	13s

3. In another terminal, get a shell to the running Container:

```
kubectl exec -it redis -- /bin/bash
```

4. In your shell, go to /data/redis, and then create a file:

```
cd /data/redis/  
echo Hello > test-file
```

5. In your shell, list the running processes:

```
apt-get update  
apt-get install procps  
ps aux
```

6. The output is similar to this:

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
redis	1	0.1	0.1	33308	3828	?	Ssl	00:46	0:00	redis-server *:6379
root	12	0.0	0.0	20228	3020	?	Ss	00:47	0:00	/bin/bash
root	15	0.0	0.0	17500	2072	?	R+	00:48	0:00	ps aux

7. In your shell, kill the Redis process:

```
root@redis:/data/redis# kill <pid>
```

where <pid> is the Redis process ID (PID).

In your original terminal, watch for changes to the Redis Pod. Eventually, you will see something like this:

NAME	READY	STATUS	RESTARTS	AGE
redis	1/1	Running	0	13s
redis	0/1	Completed	0	6m
redis	1/1	Running	1	6m

8. At this point, the Container has terminated and restarted. This is because the Redis Pod has a restartPolicy of Always.

9. Get a shell into the restarted Container:

```
kubectl exec -it redis -- /bin/bash
```

10. In your shell, go to /data/redis, and verify that test-file is still there.

```
cd /data/redis/  
ls  
test-file
```

11. Delete the Pod that you created for this exercise:

```
kubect1 delete pod redis
```

Practical Lesson 25.2: Create a Kubernetes Job

Overview

A Kubernetes job is a workload controller object that performs one or more finite tasks in a cluster. The finite nature of jobs differentiates them from most controller objects, such as deployments, replica sets, stateful sets, and daemon sets. While these objects permanently maintain the desired state and number of pods in the cluster, jobs run until they complete the task and then terminate the associated pods.

Prerequisites

- Should be connected to the internet
- OKE cluster

Tasks

1. A job in Kubernetes is created using a YAML file. Follow the steps below to deploy a Kubernetes job.
2. Create a YAML file by creating a file in code editor.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: test-job
spec:
  template:
    metadata:
      name: test-job
    spec:
      containers:
      - name: test
        image: alpine:latest
        command:
        - "bin/sh"
        - "-c"
        - "for i in 1 2 3 4 5 6 7 8 9 ; do echo $i ; done"
      restartPolicy: Never
```

The example below creates a *test-job* file. This job runs a pod with a container based on the `alpine:latest` Docker image. Inside the container, the job prints the numbers from one to nine and then terminates the container.

3. Apply the YAML file using `kubectl`:

```
kubectl apply -f [filename].yaml
```

The output confirms the successful job creation.

4. Confirm the job execution by checking the state of the pods in the cluster:

```
kubectl get pod
```

The **READY** column shows the pod is not running anymore and the pod's status is Completed.

NAME	READY	STATUS	RESTARTS	AGE
test-job-5ms7q	0/1	Completed	0	62s

5. Use **`kubectl describe`** to see pod details:

```
kubectl describe pod [pod-name]
```

The output shows the pod's status as Succeeded.

6. The **Containers** section of the output lists the container state as *Terminated*. The reason for termination is the completion of the operation.

```
Containers: ←
  test:
    Container ID:  docker://b0eaf902dbf6a9a5ed5a415ea3e93560397c76f610885fdbb20cc894a2209680
    Image:          alpine:latest
    Image ID:       docker-pullable://alpine@sha256:686d8c9dfa6f3ccfc8230bc3178d23f84eea
                    f7e457f36f271ab1acc53015037c
    Port:          <none>
    Host Port:     <none>
    Command:
      bin/sh
      -C
      for i in 1 2 3 4 5 6 7 8 9 ; do echo $i ; done
    State:         Terminated
      Reason:      Completed
      Exit Code:    0
      Started:     Tue, 28 Jun 2022 08:28:26 -0400
      Finished:    Tue, 28 Jun 2022 08:28:26 -0400
    Ready:         False
    Restart Count: 0
    Environment:   <none>
```

You can also check the job itself by typing the following command:

```
kubectl get job
```

The **COMPLETIONS** column shows the job completed successfully.

NAME	COMPLETIONS	DURATION	AGE
test-job	1/1	4s	29s

Now, you should know how to deploy and configure Kubernetes jobs

PRACTICAL LESSON 26
NAMESPACE AND RESOURCE QUOTAS

Practical Lesson 26: Overview

Practical Overview

In this Lab, you will understand how to assign memory and CPU resources to container and Pods.

Estimated Completion Time: 40mins

Practical Lesson 26.1: Assign Memory resources to Container and Pods

Overview

This practical will show you how to assign a memory *request* and a memory *limit* to a Container. A Container is guaranteed to have as much memory as it requests, but is not allowed to use more memory than its limit.

Prerequisites

- Should be connected to the internet
- Metrics Server should be installed
- OKE cluster with 2 nodes
- Minimum 1CPU

Tasks

1. Create a Namespace

```
kubectl create namespace mem-example
```

2. To specify a memory request for a Container, include the `resources:requests` field in the Container's resource manifest. To specify a memory limit, include `resources:limits`.

In this exercise, you create a Pod that has one Container. The Container has a memory request of 100 MiB and a memory limit of 200 MiB. Here's the configuration file for the Pod:

```

apiVersion: v1
kind: Pod
metadata:
  name: memory-demo
  namespace: mem-example
spec:
  containers:
  - name: memory-demo-ctr
    image: polinux/stress
    resources:
      requests:
        memory: "100Mi"
      limits:
        memory: "200Mi"
    command: ["stress"]
    args: ["--vm", "1", "--vm-bytes", "150M", "--vm-hang", "1"]

```

3. The args section in the configuration file provides arguments for the Container when it starts. The "--vm-bytes", "150M" arguments tell the Container to attempt to allocate 150 MiB of memory.

Create the Pod:

```
kubectl apply -f memory-request.yaml -n mem-example
```

4. Verify that the Pod Container is running:

```
kubectl get pod memory-demo -n mem-example
```

5. View detailed information about the Pod:

```
kubectl get pod memory-demo --output=yaml -n mem-example
```

The output shows that the one Container in the Pod has a memory request of 100 MiB and a memory limit of 200 MiB.

6. Run `kubectl top` to fetch the metrics for the pod:

```
kubectl top pod memory-demo -n mem-example
```

The output shows that the Pod is using about 162,900,000 bytes of memory, which is about 150 MiB. This is greater than the Pod's 100 MiB request, but within the Pod's 200 MiB limit.

NAME	CPU(cores)	MEMORY(bytes)
memory-demo	<something>	162856960

7. Delete your Pod:

```
kubectl delete pod memory-demo -n mem-example
```

EXCEED A CONTAINER'S MEMORY LIMIT

A Container is not allowed to use more than its memory limit. If a Container allocates more memory than its limit, the Container becomes a candidate for termination. If the Container continues to consume memory beyond its limit, the Container is terminated.

1. Create another file inside the main directory

where, you create a Pod that attempts to allocate more memory than its limit. Here is the configuration file for a Pod that has one Container with a memory request of 50 MiB and a memory limit of 100 MiB

```

apiVersion: v1
kind: Pod
metadata:
  name: memory-demo-2
  namespace: mem-example
spec:
  containers:
  - name: memory-demo-2-ctr
    image: polinux/stress
    resources:
      requests:
        memory: "50Mi"
      limits:
        memory: "100Mi"
    command: ["stress"]
    args: ["--vm", "1", "--vm-bytes", "250M", "--vm-hang", "1"]

```

2. Create the Pod:

```
kubectl apply -f memory-request-limit-2.yaml -n mem-example
```

View detailed information about the Pod:

```
kubectl get pod memory-demo-2 -n mem-example
```

At this point, the Container might be running or killed. Repeat the preceding command until the Container is killed:

NAME	READY	STATUS	RESTARTS	AGE
memory-demo-2	0/1	OOMKilled	1	24s

3. Get a more detailed view of the Container status:

```
kubectl get pod memory-demo-2 --output=yaml -n mem-example
```

The output shows that the Container was killed because it is out of memory (OOM):

```
lastState:
  terminated:
    containerID: 65183c1877aaec2e8427bc95609cc526
    exitCode: 137
    finishedAt: 2017-06-20T20:52:19Z
    reason: OOMKilled
    startedAt: null
```

4. The Container in this exercise can be restarted, so the kubelet restarts it. Repeat this command several times to see that the Container is repeatedly killed and restarted:

```
kubectl get pod memory-demo-2 -n mem-example
```

The output shows that the Container is killed, restarted, killed again, restarted again, and so on:

```
pod/memory-demo-2 created
ritesh@cloudshell:Lab (us-ashburn-1)$ kubectl get pod memory-demo-2 -n mem-example
NAME          READY   STATUS    RESTARTS   AGE
memory-demo-2 0/1     OOMKilled 2 (15s ago) 18s
ritesh@cloudshell:Lab (us-ashburn-1)$ kubectl get pod memory-demo-2 -n mem-example
NAME          READY   STATUS             RESTARTS   AGE
memory-demo-2 0/1     CrashLoopBackOff   3 (21s ago) 66s
ritesh@cloudshell:Lab (us-ashburn-1)$ kubectl get pod memory-demo-2 -n mem-example
NAME          READY   STATUS             RESTARTS   AGE
memory-demo-2 0/1     CrashLoopBackOff   4 (25s ago) 116s
ritesh@cloudshell:Lab (us-ashburn-1)$
```

5. View detailed information about the Pod history:

```
kubectl describe pod memory-demo-2 -n mem-example
```

The output shows that the Container starts and fails repeatedly:

```
Normal    Pulled      2m37s          kubelet          Successfully pulled image "polinux/stress" in 143.40639ms (143.424986ms including waiting)
Normal    Pulled      2m23s          kubelet          Successfully pulled image "polinux/stress" in 241.171425ms (241.273991ms including waiting)
Normal    Created     115s (x4 over 2m38s) kubelet          Created container memory-demo-2-ctr
Normal    Started     115s (x4 over 2m38s) kubelet          Started container memory-demo-2-ctr
Normal    Pulled      115s          kubelet          Successfully pulled image "polinux/stress" in 164.484676ms (164.495486ms including waiting)
Warning   BackOff     83s (x7 over 2m36s) kubelet          Back-off restarting failed container memory-demo-2-ctr in pod memory-demo-2_mem-example(b9b4695
Normal    Pulling     69s (x5 over 2m39s) kubelet          Pulling image "polinux/stress"
Normal    Pulled      69s          kubelet          Successfully pulled image "polinux/stress" in 198.319488ms (198.337693ms including waiting)
ritesh@cloudshell:Lab (us-ashburn-1)$ kubectl get pod memory-demo-2 -n mem-example
NAME          READY   STATUS    RESTARTS   AGE
memory-demo-2 0/1     OOMKilled 5 (92s ago) 3m3s
ritesh@cloudshell:Lab (us-ashburn-1)$
```

6. Delete your Pod:

```
kubectl delete pod memory-demo-2 -n mem-example
```

Practical Lesson 26.2: Assign CPU resources to Container and Pods

Overview

This practical will show you how to assign a CPU *request* and a CPU *limit* to a container. Containers cannot use more CPU than the configured limit. Provided the system has CPU time free, a container is guaranteed to be allocated as much CPU as it requests

Prerequisites

- Should be connected to the internet
- OKE cluster with 2 nodes
- Minimum 1CPU
- Metrics Server should be installed

Tasks

1. CREATE A NAMESPACE

Create a Namespace so that the resources you create in this lab are isolated from the rest of your cluster.

```
kubectl create namespace cpu-example
```

2. SPECIFY A CPU REQUEST AND A CPU LIMIT

To specify a CPU request for a container, include the `resources:requests` field in the Container resource manifest. To specify a CPU limit, include `resources:limits`.

In this exercise, you create a Pod that has one container. The container has a request of 0.5 CPU and a limit of 1 CPU. Here is the configuration file for the Pod:


```

apiVersion: v1
kind: Pod
metadata:
  name: cpu-demo
  namespace: cpu-example
spec:
  containers:
  - name: cpu-demo-ctr
    image: vish/stress
    resources:
      limits:
        cpu: "1"
      requests:
        cpu: "0.5"
    args:
    - -cpus
    - "2"

```

The args section of the configuration file provides arguments for the container when it starts. The -cpus "2" argument tells the Container to attempt to use 2 CPUs.

3. Create the Pod:

```
kubectl apply -f cpu-request-limit.yaml -n cpu-example
```

4. Verify that the Pod is running:

```
kubectl get pod cpu-demo -n cpu-example
```

View detailed information about the Pod:

```
kubectl get pod cpu-demo --output=yaml -n cpu-example
```

The output shows that the one container in the Pod has a CPU request of 500 milliCPU and a CPU limit of 1 CPU.

```
resources:
  limits:
    cpu: "1"
  requests:
    cpu: 500m
```

5. Use `kubectl top` to fetch the metrics for the Pod:

```
kubectl top pod cpu-demo -n cpu-example
```

This example output shows that the Pod is using 974 milliCPU, which is slightly less than the limit of 1 CPU specified in the Pod configuration.

NAME	CPU(cores)	MEMORY(bytes)
cpu-demo	974m	<something>

6. Delete your Pod:

```
kubectl delete pod cpu-demo -n cpu-example
```

SPECIFY A CPU REQUEST THAT IS TOO BIG FOR YOUR NODES

CPU requests and limits are associated with Containers, but it is useful to think of a Pod as having a CPU request and limit. The CPU request for a Pod is the sum of the CPU requests for all the Containers in the Pod. Likewise, the CPU limit for a Pod is the sum of the CPU limits for all the Containers in the Pod.

Now, you create a Pod that has a CPU request so big that it exceeds the capacity of any Node in your cluster. Here is the configuration file for a Pod that has one Container. The Container requests 100 CPU, which is likely to exceed the capacity of any Node in your cluster.

```
apiVersion: v1
kind: Pod
metadata:
  name: cpu-demo-2
  namespace: cpu-example
spec:
  containers:
  - name: cpu-demo-ctr-2
    image: vish/stress
    resources:
      limits:
        cpu: "100"
      requests:
        cpu: "100"
    args:
    - -cpus
    - "2"
```

1. Create the Pod:

```
kubectl apply -f cpu-request-limit-2.yaml -n cpu-example
```

2. View the Pod status:

```
kubectl get pod cpu-demo-2 -n cpu-example
```

The output shows that the Pod status is Pending. That is, the Pod has not been scheduled to run on any Node, and it will remain in the Pending state indefinitely:

NAME	READY	STATUS	RESTARTS	AGE
cpu-demo-2	0/1	Pending	0	7m

3. View detailed information about the Pod, including events:

```
kubectl describe pod cpu-demo-2 -n cpu-example
```

The output shows that the Container cannot be scheduled because of insufficient CPU resources on the Nodes:

```

    ConfigMapName:      kube-root-ca.crt
    ConfigMapOptional:  <nil>
    DownwardAPI:        true
  QoS Class:           Burstable
  Node-Selectors:      <none>
  Tolerations:         node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                      node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type     Reason          Age   From          Message
  ----     -
Warning   FailedScheduling 37s   default-scheduler  0/2 nodes are available: 2 Insufficient cpu.
preemption: 0/2 nodes are available: 2 No
preemption victims found for incoming pod..
ritesh@cloudshell:lab (us-ashburn-1)$

```

4. Delete your Pod:

```
kubectl delete pod cpu-demo-2 -n cpu-example
```

Practical 26.3: ASSIGN PODS TO NODES

Overview

In this practical, we'll learn how to assign a Kubernetes Pod to a particular node in a Kubernetes cluster.

Pre-requisites

- Should be connected to the internet.
OKE cluster with 2 nodes

BEFORE YOU BEGIN

You need to have a Kubernetes cluster, and the `kubectl` command-line tool must be configured to communicate with your cluster. It is recommended to run this practical on a cluster with at least two nodes that are not acting as control plane hosts.

Add a label to a node

1. List the nodes in your cluster:

```
kubectl get nodes
```

Copy the IP address of the node on which you want to schedule your pod

2. Choose one of your nodes, and add a label to it:

```
kubectl label nodes <your-node-name> disktype=ssd
```

where `<your-node-name>` is the name of your chosen node.

Verify that your chosen node has a `disktype=ssd` label:

```
kubectl get nodes --show-labels
```

CREATE A POD THAT GETS SCHEDULED TO YOUR CHOSEN NODE

This pod configuration file describes a pod that has a node selector, `disktype: ssd`. This means that the pod will get scheduled on a node that has a `disktype=ssd` label.

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  nodeName: 10.0.10.250 # schedule pod to specific node
  containers:
  - name: nginx
    image: nginx
    imagePullPolicy: IfNotPresent
```

Use the configuration file to create a pod that will get scheduled on your chosen node:

```
kubectl create -f pod.yaml
```

Verify that the pod is running on your chosen node:

```
kubectl get pods --output=wide
```

The output is similar to this:

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
nginx	1/1	Running	0	13s	10.200.0.4	worker0

You have successfully assigned a pod to a particular pod.

PRACTICAL LESSON 27
OKE ARCHITECTURE

Practical Lesson 27: Overview

Practical Overview

There are no practicals for this lesson.

END OF THE DOCUMENT